

Benchmarking Retrieval Strategies for Long-Document Summarization

Ilaha Karimli

Izmir Institute of Technology, Department of Computer Engineering, Izmir, Türkiye
ilahakarimli06@gmail.com

Abstract. Long-document summarization must balance quality and computational efficiency, yet existing methods are often evaluated in isolation, offering limited guidance under resource constraints. We present a systematic benchmark of efficiency–quality trade-offs across widely used long-document summarization strategies, including sliding-window processing and retrieval-based pipelines. Experiments are conducted on arXiv and Longform, spanning documents from 4K to 32K tokens. Under consistent settings, we evaluate summary quality, retrieval accuracy, latency, throughput, and resource usage. Chunk-based retrieval substantially outperforms sliding-window methods on extra-long documents, achieving over $2\times$ ROUGE-L gains while reducing latency. ColBERT achieves strong retrieval effectiveness but incurs large index sizes, while PLAID reduces index memory by up to $365\times$ with modest recall degradation and millisecond-level latency. FAISS-based compression yields significant memory savings with minimal impact on summarization quality. Overall, we provide a unified and reproducible comparison of long-document summarization pipelines under realistic efficiency constraints, guiding memory- and latency-sensitive deployment.

Keywords: long-document summarization · retrieval-augmented generation · efficiency–quality trade-offs · ColBERT · PLAID

1 Introduction

Abstractive summarization of long documents is constrained by the fixed context limits of transformer-based models, often requiring truncation or segmentation that leads to information loss and higher inference cost.[1,2] Larger context windows help, but are limited by self-attention’s quadratic complexity [3]. Prior work applies inference-time strategies to extend context or process document segments; sparse-attention and retrieval-based pipelines improve scalability but add architectural complexity or rely on heuristics. Despite their widespread use, these strategies are rarely compared under identical conditions, making it difficult to isolate the effects of model capacity, context length, and retrieval or compression design choices. In particular, compression methods are typically evaluated using retrieval metrics, leaving their downstream impact underexplored.

We present a controlled comparison of inference-time strategies for long-document summarization. We evaluate sliding-window and retrieval-based pipelines,

encompassing both dense and token-level methods, within a unified framework using identical datasets (arXiv [4] and Longform [5]), fixed context budgets, and consistent preprocessing. We evaluate documents up to 32K tokens, analyzing trade-offs among summarization quality, latency, throughput, and memory. By isolating inference-time effects (no fine-tuning), we provide reproducible guidance for selecting long-document summarization pipelines under computational constraints.

2 Related Work

Prior work on long-document abstractive summarization has explored diverse inference-time strategies to address the limitations of transformer context length.

Sparse or structured attention extends transformers for long-context processing. Longformer and its encoder-decoder variant LED, along with BigBird and LongT5, achieve near-linear complexity while preserving document-level context [1,2,3]. Earlier hierarchical methods (e.g., discourse-aware summarization, HiBERT) use multi-stage aggregation but are rarely compared under identical constraints. HEPOS [6] enables longer inputs, yet architectural complexity and limited ultra-long evaluation hinder comparison.

Retrieval-based pipelines, such as RAG and dense chunk-based methods, select relevant segments for summarization [7,8,9]. These works often fix design choices, limiting trade-off analysis. Li et al. [10] show that long-context LLMs perform strongly with sufficient resources; however, efficiency-focused evaluations of retrieval-based methods remain limited in summarization studies.

Token-level methods such as ColBERT and PLAID [11,12] offer strong retrieval accuracy through late interaction, but are rarely studied in end-to-end summarization. Overall, no prior work provides a unified, controlled comparison of sliding-window, retrieval-based, and token-level inference-time strategies while jointly analyzing summarization quality, efficiency, and compression.

3 Methodology

3.1 Task and Scope Definition

We examine inference-time strategies for long-document abstractive summarization under fixed context windows, focusing on single-document settings and excluding fine-tuning to isolate long-context handling effects.

Table 1: Method Families and Core Characteristics

Method	Strategy	Models	Retrieval
Sliding Window	Overlap + aggregation	LED, BigBird, FLAN-T5	None
Chunk Retrieval	Dense chunk selection	LED, BigBird, LongT5	Chunk
Dense Index	FAISS index variants	BigBird + FAISS	Chunk
Token-Level	Late interaction (MaxSim)	BigBird + ColBERT	Token
Compressed	Pruned token retrieval	BigBird + PLAID	Token

3.2 Methods Under Comparison

We compare five inference-time strategies, summarized in Table 1 and Table 2.

Table 2: Key Variations per Method Family

Method	Variations
Sliding Window	Window size, overlap, aggregation (concat/hier.)
Chunk Retrieval	Segmentation, embedding (MiniLM, BGE), top- K
Dense Index	Index type (Flat, IVF, IVF-PQ)
Token-Level	Fixed config (baseline)
Compressed	Compression ratio (M), tokens per chunk

3.3 Data and Evaluation Setup

We evaluate all methods on two datasets: arXiv scientific papers [4] and Longform articles [5]. Documents are grouped into three length ranges (4K–8K, 8K–16K, and 16K–32K tokens).

Evaluations are conducted across length categories and domains. The arXiv dataset includes structured abstracts, while Longform articles cover narrative-style content.

3.4 Controlled Experimental Design

All experiments follow a controlled setup with a fixed maximum context length of 16K tokens, generation lengths of 256–512 tokens, beam size 4, and a random seed of 42. Each configuration is evaluated on 60 randomly sampled documents.

All methods share identical preprocessing, tokenization, query generation, and decoding settings, and are executed under the same computational constraints. Within each method category, key hyperparameters are fixed: chunk-based retrieval uses a chunk size of 1024 tokens with top- $K = 3$, FAISS-based methods apply consistent retrieval settings, and compression-based approaches follow predefined parameter grids.

3.5 Evaluation Metrics and Analysis Dimensions

Summarization quality is evaluated using ROUGE-L [13] and BERTScore F1 [14]. Retrieval performance is measured with Recall@ K ($K \in 1, 5, 10$) and Mean Reciprocal Rank (MRR). For ColBERT [11] and PLAID [12], Recall@20 is additionally reported. Efficiency is measured in terms of end-to-end latency, throughput, peak GPU memory usage, and index size. Results are analyzed across quality–efficiency trade-offs, compression effects, document length robustness, and cross-domain performance.

3.6 Ablation and Sensitivity Analysis Protocol

We conduct ablation and sensitivity analyses to assess the impact of individual design choices. For sliding-window methods, we vary window size (512–2048

tokens), overlap ratio (0–50%), number of global attention tokens (0–64), and aggregation strategy (concatenation vs. hierarchical). These variations are evaluated across LED, BigBird, and FLAN-T5 under consistent windowing settings.

For retrieval-based methods, we ablate the segmentation strategy, embedding model, index type (Flat, IVF, IVF-PQ), and compression parameters. PLAID compression is evaluated over ratios ranging from approximately $100\times$ to $365\times$.

All ablations follow a single-variable protocol, where one parameter is varied while all others are held constant.

3.7 Reproducibility and Implementation Details

All experiments are implemented in PyTorch ($\geq 2.9.0$) using the Transformers ($\geq 4.57.1$) and FAISS ($\geq 1.12.0$) libraries, and run on CUDA-enabled GPUs (A100/H100). Deterministic execution is ensured by fixing random seeds across Python, NumPy, and PyTorch. All model identifiers, configuration files, and experiment scripts are released publicly, with cached embeddings and indices provided to support full reproducibility.

4 Results and Discussion

4.1 Sliding Window Analysis

Tables 3 and Figure 1 show that window size is the primary driver of summarization quality. Using 2048-token windows yields over 130% ROUGE-L gains compared to 512-token baselines across datasets. Larger windows better capture long-range dependencies, while increased overlap degrades performance by introducing redundant chunks that harm coherence.

Document length strongly affects performance (Table 4), with ROUGE-L dropping by roughly 40% from medium (4 - 8K) to extra-long (16K+) documents.

Table 3: Impact of Window Size on ROUGE-L Performance

Dataset	Window Size	LED ROUGE-L	BigBird ROUGE-L	Improvement
ArXiv	512	0.038	0.030	baseline
ArXiv	2048	0.089	0.079	+134%
Longform	512	0.037	0.035	baseline
Longform	2048	0.080	0.085	+130%

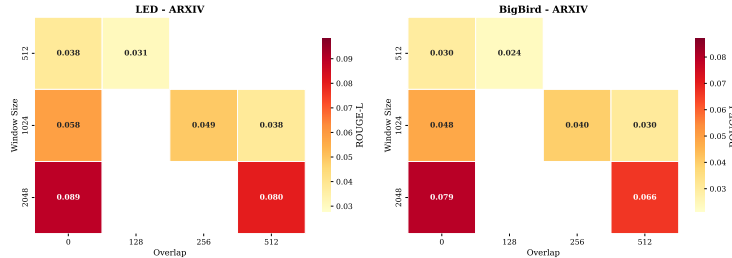


Fig. 1: ROUGE-L heatmap for LED and BigBird on the ArXiv dataset across different window sizes and overlap values.

BigBird performs better on shorter inputs (0.163 vs. 0.139), while LED is more robust on longer documents (0.089 vs. 0.079). This reflects architectural differences: BigBird benefits from global attention at moderate lengths, whereas LED’s local attention scales more effectively to very long sequences. These trends are consistent across both ArXiv and Longform datasets (Table 5).

Table 4: Document Length & ROUGE-L(Window Size = 2048, Overlap = 0)

Dataset	Length	LED ROUGE-L	BigBird ROUGE-L
ArXiv	Medium	0.139	0.163
	Long	0.119	0.116
	Extra-Long	0.089	0.079
Longform	Medium	0.127	0.161
	Long	0.106	0.121
	Extra-Long	0.080	0.085

Table 5: LED vs. BigBird Comparison (Overlap = 0, Global Tokens = 0)

Dataset	Metric	LED	BigBird
ArXiv	ROUGE-L	0.089	0.079
	BERTScore	0.805	0.838
	Latency (s)	2.19	7.41
	GPU Memory (GB)	5.09	8.43
Longform	ROUGE-L	0.080	0.085
	BERTScore	0.785	0.832
	Latency (s)	2.22	7.45
	GPU Memory (GB)	5.08	8.47

Under optimal settings (Table 5), LED provides $3.4\times$ faster inference and 40% lower GPU memory with comparable quality, due to its local attention. Hierarchical aggregation boosts BigBird’s ROUGE-L by 420% (Table 6) by compressing chunk-level information before global synthesis. BigBird preserves BERTScore (0.84), while LED shows slight semantic degradation, highlighting the benefit of global attention in second-stage summarization.

Table 6: Aggregation Comparison (Window =2048, Extra-Long Documents)

Dataset	Model	Strategy	ROUGE-L	BERTScore	Δ ROUGE	Δ BERT
ArXiv	LED	Concat	0.049	0.819	—	—
ArXiv	LED	Hierarchical	0.052	0.748	+6%	-9%
ArXiv	BigBird	Concat	0.040	0.838	—	—
ArXiv	BigBird	Hierarchical	0.207	0.840	+420%	+0.2%

4.2 Chunk Retrieval Analysis

Table 7 compares models under retrieval-augmented generation. BigBird achieves the highest ROUGE-L (0.245 on ArXiv, 0.218 on Longform), outperforming LED and LongT5 by over 50%. This stems from BigBird’s global attention, which

more effectively synthesizes retrieved chunks, whereas LED’s local attention struggles with disjoint content. LongT5 underperforms despite its long-sequence design, likely due to weaker integration of concatenated retrieved chunks.

Segmentation strategy further affects performance (Table 8). Semantic segmentation yields the best ROUGE-L (0.173) and lowest latency (0.89s) by preserving discourse boundaries, while fixed chunking underperforms sentence-level segmentation. It also reduces GPU memory usage by 15%, providing both quality and efficiency gains.

Table 7: Chunk Retrieval Performance (Chunk Size = 1024, Top-K = 3)

ArXiv			
Metric	BigBird	LED	LongT5
ROUGE-L	0.245	0.157	0.164
BERTScore	0.850	0.816	0.813
Latency (s)	3.39	1.94	4.83
GPU (GB)	3.35	2.25	2.50
Longform			
Metric	BigBird	LED	LongT5
ROUGE-L	0.218	0.146	0.148
BERTScore	0.844	0.803	0.806
Latency (s)	3.10	1.95	4.78
GPU (GB)	3.30	2.22	2.44

Table 8: Segmentation Strategy on Chunk Retrieval

Metric	Fixed	Sentence	Semantic
ROUGE-L	0.152	0.160	0.173
BERTScore	0.816	0.821	0.830
Latency (s)	1.35	1.35	0.89
GPU (GB)	1.92	1.81	1.63

Table 9: Embedding Comparison (Big-Bird, ArXiv, Chunk Size = 1024)

Metric	BGE-base	MiniLM-L6
ROUGE-L	0.245	0.240
BERTScore	0.850	0.850
Δ ROUGE	–	-2.4%

4.3 Dense Retrieval Analysis

Table 10 compares FAISS index configurations. IVF-PQ achieves $37\times$ compression (14.27MB to 0.38MB with 4-bit quantization) while maintaining comparable ROUGE-L (0.234 vs. 0.222), indicating that aggressive quantization does not harm downstream summarization. IVF indexing further reduces retrieval latency by $8\times$ without quality loss, showing that approximate search is sufficient.

Table 10: FAISS Index Type Comparison (BGE Embedding, ArXiv)

Index Type	Index Size (MB)	Recall@10	Latency (ms)	ROUGE-L
Flat	14.27	0.41	1.28	0.222
IVF (nlist=64)	14.50	0.41	0.16	0.220
IVF-PQ (4bit)	0.38	0.37	0.12	0.234
IVF-PQ (8bit)	1.20	0.41	0.29	0.219

Despite similar recall@10 across index types (~ 0.37 – 0.41), ROUGE-L remains stable (~ 0.22 – 0.23), suggesting that summarization models compensate for retrieval imprecision. Overall, IVF-PQ offers the best efficiency–quality trade-off and is therefore preferred for deployment.

4.4 Token-Level Retrieval Analysis

Table 11 reports ColBERT’s token-level retrieval results. Although ColBERT achieves lower recall than dense retrieval (Recall@10 = 0.46 on ArXiv, 0.36 on Longform), it attains comparable downstream ROUGE-L (0.237 vs. 0.222), indicating that token-level matching captures semantically relevant content despite weaker exact retrieval. This comes at a high efficiency cost: ColBERT requires a $230\times$ larger index and $760\times$ slower retrieval due to per-token embeddings. Given the marginal quality gain ($<7\%$), dense retrieval with IVF-PQ offers a more favorable efficiency–quality trade-off for long-document summarization.

Table 11: ColBERT Token-Level Retrieval

	ArXiv	Longform
Recall@10	0.46	0.36
ROUGE-L	0.237	0.215
Index (MB)	88.5	88.4
Latency (ms)	121.5	124.3
GPU (GB)	3.58	3.50

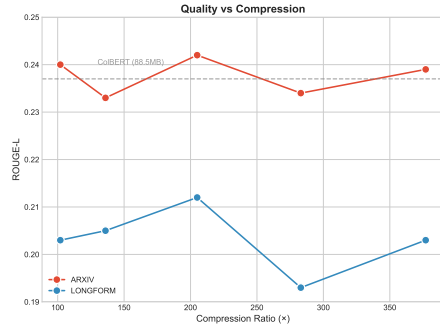


Fig. 2: PLAID compression analysis: compression ratio vs. ROUGE-L

4.5 Compressed Token-Level Retrieval (PLAID)

Table 12 and Figure 2 analyze PLAID’s token pruning and PQ compression. PLAID achieves up to $365\times$ index compression over ColBERT with comparable ROUGE-L, indicating that aggressive token pruning preserves semantic relevance. The reduced index yields over $7\times$ lower retrieval latency. Varying token counts per chunk has minimal impact on quality, suggesting redundancy in token representations. Overall, PLAID offers the best quality–efficiency trade-off under storage and latency constraints.

Table 12: PLAID Compression Configurations (ArXiv)

Tokens/Chunk	M	Compression	Index (MB)	Latency (ms)	ROUGE-L
64	96	102 $\times$	0.89	15.9	0.240
32	96	205 $\times$	0.44	15.0	0.242
32	48	283 $\times$	0.32	17.5	0.234
24	48	377 $\times$	0.24	16.8	0.239

5 Conclusion

This work presents a unified comparison of inference-time strategies for long-document summarization under realistic computational constraints. Our results

show that retrieval-based pipelines consistently outperform sliding-window approaches on extra-long documents, offering improved quality with lower latency and memory usage. Compression-aware retrieval methods further enable substantial reductions in index size with minimal impact on summarization quality, making them particularly suitable for resource-constrained settings. Overall, our findings indicate that compressed retrieval-based pipelines provide the most practical and scalable solution for long-document summarization.

References

1. I Beltagy et al. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
2. M Zaheer et al. Big bird: Transformers for longer sequences. *arXiv preprint arXiv:2007.14062*, 2020.
3. M Guo. Longt5: Efficient text-to-text transformer for long sequences. In *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022.
4. Ccdv Research Group. arxiv summarization dataset. <https://huggingface.co/datasets/ccdv/arxiv-summarization>, 2023.
5. V. Goldberg. Longform article summarization dataset. https://huggingface.co/datasets/vgoldberg/longform_article_summarization, 2023.
6. L Huang et al. Efficient attentions for long document summarization. In *Proceedings of NAACL-HLT 2021*. Association for Computational Linguistics, 2021.
7. P Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
8. W Xiao et al. Extractive summarization of long documents by combining global and local context. In *Proceedings of EMNLP-IJCNLP 2019*. Association for Computational Linguistics, 2019.
9. C An et al. Retrievalsum: A retrieval enhanced framework for abstractive summarization. *arXiv preprint arXiv:2109.07943*, 2021.
10. Z Li et al. Retrieval augmented generation or long-context llms? a comprehensive study and hybrid approach. 2024.
11. O Khattab et al. Colbert: Efficient and effective passage search via contextualized late interaction. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.
12. K Santhanam et al. Plaid: An efficient engine for late interaction retrieval. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*, 2022.
13. Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, 2004.
14. T Zhang et al. Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations*, 2020.